

Project Design Phase-II

Technology Stack (Architecture & Stack)

Date	19 July 2026
Team ID	[Enter Your Team ID]
Project Name	BookEase – Multi-Role Online Bookstore Platform
Maximum Marks	4 Marks

Technical Architecture

BookEase follows a classic 3-tier MERN architecture: a React single-page application (presentation), an Express/Node REST API (application logic), and MongoDB (data), as shown below.

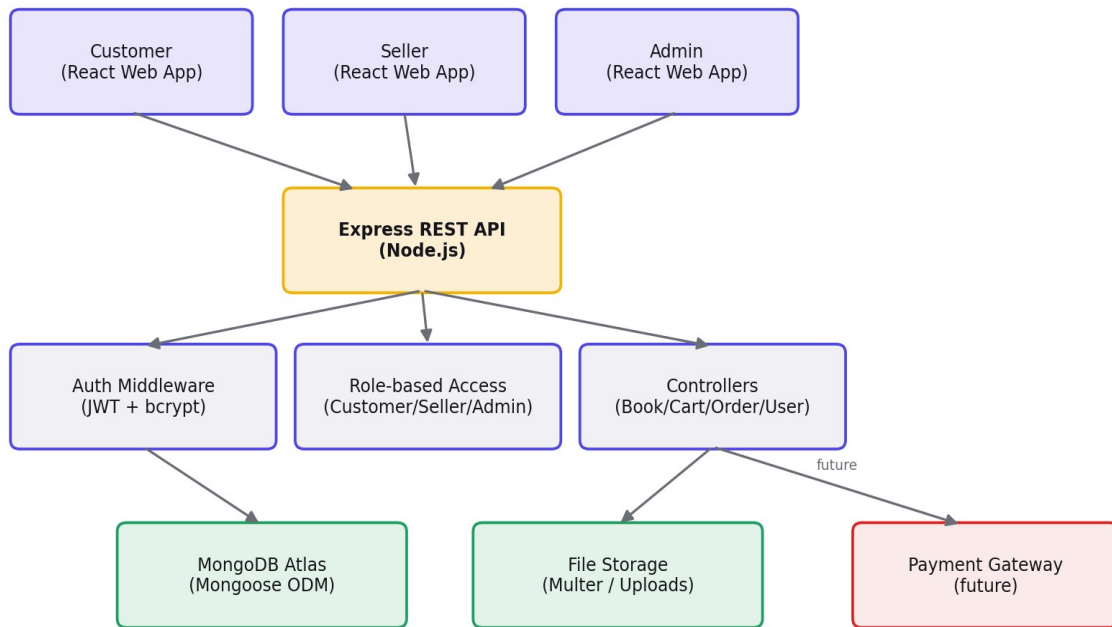


Table-1: Components & Technologies

S.No	Component	Description	Technology
1	User Interface	How the user interacts with the application — web UI for customers, sellers and admins	React.js, React Router, CSS
2	Application Logic-1	Authentication & authorization	JWT (jsonwebtoken), bcrypt
3	Application Logic-2	Business logic for books, cart,	Node.js, Express.js (REST

S.No	Component	Description	Technology
		wishlist, orders, users	controllers)
4	Application Logic-3	HTTP client & API communication from the frontend	Axios (with interceptors for auth tokens)
5	Database	Data type, schema & configuration	MongoDB (Mongoose ODM)
6	Cloud Database	Managed database service	MongoDB Atlas
7	File Storage	Storage for cover images & profile pictures	Multer (local uploads); Cloudinary planned for production persistence
8	External API-1 (planned)	Online payment collection	Razorpay / Stripe
9	External API-2 (planned)	Transactional email for password reset & order confirmation	Nodemailer / SendGrid
10	State Management	Client-side auth & cart state, with localStorage caching	React Context API
11	Infrastructure (Server / Cloud)	Application deployment	Backend: Render/Railway. Frontend: Vercel/Netlify. Database: MongoDB Atlas

Table-2: Application Characteristics

S.No	Characteristics	Description	Technology
1	Open-Source Frameworks	Core frameworks and libraries used across the stack	React, Express, Mongoose, bcrypt, JWT, Axios
2	Security Implementations	Access control and data protection measures	JWT auth, bcrypt password hashing, role-based middleware (protect / adminOnly / sellerOnly), server-side input validation
3	Scalable Architecture	3-tier design keeps presentation, application, and data layers independently deployable	Stateless REST API + MongoDB Atlas replica set
4	Availability	Multiple Node.js instances can run behind the hosting provider's load balancer	Render/Railway auto-restart & managed uptime
5	Performance	Pagination, indexed search, and client-side cart caching reduce load and latency	MongoDB text index, localStorage cache, lean Mongoose queries